

Search
Home
GitHub
CLASSES
ArticlesService
ErrorBoundary
EVENTS
SIGINT
unhandledRejection
NAMESPACES
apiConfig
contactoEmergenciaService
diarioService
GLOBAL
404_Error_Handler
ACTIVIDADES
API_URL
ASPECTOS_COGNITIVOS
ActionCard
ActivitySelector
App
Articulos
AuthButtons
AuthLayout
Button
CORS_Configuration
Card
Carousel
Checkbox
CircularProgress
CognitionSelector
Diario
DiaryBanner
DiaryEditor
DiaryEntry
EMOCIONES
EmergencyButton
EmergencyModal
EmotionSelector
EmotionStates

frontend/src/hooks/useToast.js

```
1.  /**
2.   * @file Hook personalizado para mostrar notificaciones (toasts).
3.   * @description Proporciona funciones para mostrar fácilmente notifi
4.   * @requires react-hot-toast
5.   */
6.  import toast from 'react-hot-toast';
7.
8.  /**
9.   * @function useToast
10.   * @description Un hook personalizado que abstrae la librería `react
11.   * @returns {object} Un objeto con funciones para mostrar diferentes
12.   * @property {function} success - Muestra un toast de éxito.
13.   * @property {function} error - Muestra un toast de error.
14.   * @property {function} info - Muestra un toast de información.
15.   * @property {function} loading - Muestra un toast de carga.
16.   * @property {function} dismiss - Oculta un toast específico.
17.   * @property {function} promise - Muestra toasts automáticamente par
18.   */
19.  export const useToast = () => {
20.    /**
21.     * @function success
22.     * @description Muestra un toast de éxito.
23.     * @param {string} message - El mensaje a mostrar.
24.     * @param {object} [options] - Opciones adicionales para el toas
25.     */
26.    const success = (message, options = {}) => {
27.      toast.success(message, {
28.        duration: 3000,
29.        position: 'top-right',
30.        ...options
31.      });
32.    };
33.
34.    /**
35.     * @function error
36.     * @description Muestra un toast de error.
37.     * @param {string} message - El mensaje a mostrar.
38.     * @param {object} [options] - Opciones adicionales para el toas
39.     */
40.    const error = (message, options = {}) => {
41.      toast.error(message, {
42.        duration: 4000,
43.        position: 'top-right',
44.        ...options
45.      });
46.    };
47.
48.    /**
49.     * @function info
50.     * @description Muestra un toast de información.
51.     * @param {string} message - El mensaje a mostrar.
52.     * @param {object} [options] - Opciones adicionales para el toas
```

```

53.      */
54.      const info = (message, options = {}) => {
55.          toast(message, {
56.              icon: 'i',
57.              duration: 3000,
58.              position: 'top-right',
59.              ...options
60.          });
61.      };
62.
63.      /**
64.       * @function loading
65.       * @description Muestra un toast de carga que puede ser actualiz
66.       * @param {string} message - El mensaje a mostrar.
67.       * @returns {string} El ID del toast de carga.
68.       */
69.      const loading = (message) => {
70.          return toast.loading(message, {
71.              position: 'top-right'
72.          });
73.      };
74.
75.      /**
76.       * @function dismiss
77.       * @description Oculta un toast específico por su ID.
78.       * @param {string} toastId - El ID del toast a ocultar.
79.       */
80.      const dismiss = (toastId) => {
81.          toast.dismiss(toastId);
82.      };
83.
84.      /**
85.       * @function promise
86.       * @description Maneja una promesa y muestra toasts de carga, éx
87.       * @param {Promise<any>} promise - La promesa a manejar.
88.       * @param {object} messages - Los mensajes para cada estado de l
89.       * @param {string} [messages.loading] - Mensaje mientras la prom
90.       * @param {string} [messages.success] - Mensaje cuando la prome
91.       * @param {string} [messages.error] - Mensaje cuando la promesa
92.       * @returns {Promise<any>} La promesa original.
93.       */
94.      const promise = (promise, messages) => {
95.          return toast.promise(
96.              promise,
97.              {
98.                  loading: messages.loading || 'Cargando...',
99.                  success: messages.success || '¡Éxito!',
100.                  error: messages.error || 'Error'
101.              },
102.              {
103.                  position: 'top-right'
104.              }
105.          );
106.      };
107.
108.      return { success, error, info, loading, dismiss, promise };
109.  };

```